

UNIVERSIDADE FEDERAL DE OURO PRETO

INTELIGÊNCIA ARTIFICIAL
CSI701

Agente Inteligente em Labirinto

Busca Clássica, Busca Local e Busca Online

Autor:
Gabriel Henrique Fonseca Castro

8 de junho de 2026

1 Introdução

Este trabalho tem como objetivo principal o desenvolvimento e a análise de um agente inteligente projetado para atuar em um ambiente de labirinto, modelado formalmente como uma matriz de estados discretos. Este mesmo domínio será utilizado para estudar três formas diferentes de resolução de problemas em Inteligência Artificial:

- Busca Clássica;
- Busca Local;
- Busca Online.

A metodologia adotada para este estudo envolve não apenas a implementação técnica dos algoritmos em linguagem *Python*, mas também uma modelagem formal, a análise experimental do desempenho de cada abordagem, a visualização dos resultados obtidos e uma reflexão crítica acerca da aplicabilidade e das limitações das ferramentas de IA generativa no processo de desenvolvimento de software e pesquisa científica.

2 Modelagem PEAS e Classificação

Nesta seção, definem-se as características do agente inteligente e do ambiente do labirinto utilizando a modelagem PEAS (*Performance, Environment, Actuators, Sensors*). Considerando que a dinâmica de atuação do agente inteligente varia substancialmente entre as abordagens de busca clássica, local e online, a modelagem PEAS foi estruturada em três instâncias independentes. Este refinamento permite que as definições de desempenho, sensores e a classificação sejam especificadas em função da observabilidade e dos objetivos de cada cenário, conferindo robustez à análise experimental apresentada.

2.1 Busca Clássica

- **Performance:**

$$J = 100 \cdot \left(\frac{C_{\text{ótimo}}}{Erros} \right)$$

Onde $Erros = custo + 0.1 \cdot ns_{\text{expandidos}} + 0.5 \cdot clulas_{\text{revisitadas}} + 1000 \cdot tempo(ms)$.

- **Environment:** Labirinto caracterizado como um espaço determinístico, estático, discreto e totalmente observável. Contém:

- Células livres;
- Paredes;
- Posição inicial;
- Posição final.

- **Actuators:** Mover para Cima, Baixo, Esquerda, Direita.

$$A = \{\text{cima, baixo, esquerda, direita}\}$$

- **Sensors:** Acesso direto e completo à matriz representativa do labirinto, possuindo conhecimento da posição de todas as paredes e da saída previamente.
- **Classificação:** Agente baseado em objetivos.
O agente opera sob total observabilidade do ambiente e possui um estado-meta estritamente definido (alcançar o objetivo). Utilizam-se algoritmos de busca para projetar sequências de ações futuras, selecionando o caminho ótimo ou subótimo antes da execução do primeiro movimento físico.

2.2 Busca Local

- **Performance:**

$$J = 100 \cdot \left(\frac{C_{\text{ótimo}}}{\text{Erros}} \right)$$

Onde $\text{Erros} = 1000 \cdot \text{tempo} + 0.1 \cdot \text{iteraes} + \text{custo}$

E o custo ótimo é dado pelo algoritmo de busca clássica A^* .

- **Environment:** Labirinto caracterizado como um espaço determinístico, estático e discreto. Contém:

- Células livres;
- Paredes;
- Pontos de Coleta;
- Posição inicial;
- Posição final.

- **Actuators:** Operadores de modificação de rota. O agente não se move fisicamente nesta camada de abstração, mas atua alterando a sequência de visitação dos pontos de coleta.

$$A = \{\text{trocar dois pontos de coleta}\}$$

- **Sensors:** Conhecimento do custo exato do menor caminho possível entre quaisquer dois pontos de coleta, entrada e ponto de coleta, ponto de coleta e saída.

- **Classificação:** Agente baseado em utilidade.

Para este agente, o conceito de "Objetivo" tradicional não faz sentido. Qualquer ordem que passe por todos os pontos (ex: C1, C2, C3 ou C3, C1, C2) é um estado final válido. O trabalho dele não é encontrar a saída, é julgar a qualidade de um estado. A função calcula_custo que criamos atua como a sua Função de Utilidade. Ele avalia vizinhos e escolhe pular para aqueles que maximizam a sua "felicidade" (neste caso, que minimizam a distância total da rota).

Neste método de busca, não há um "objetivo" direcional simples. Qualquer permutação que englobe todos os pontos de coleta configura um estado final válido. A inteligência reside na avaliação da qualidade do estado. A função de custo projetada atua como Função de Utilidade, permitindo ao agente avaliar permutações vizinhas e adotar aquelas que minimizem o custo total do trajeto. O custo de uma solução é:

$$C(s) = d(A, C_{\pi(1)}) + \sum_{i=1}^{k-1} d(C_{\pi(i)}, C_{\pi,(i+1)}) + d(C_{\pi(k)}, B)$$

onde $d(X,Y)$ é o custo do menor caminho entre dois pontos do labirinto calculado por A^* .

2.3 Busca Online

- **Performance:**

$$J = 100 \cdot \left(\frac{C_{\text{ótimo}}}{\text{Erros}} \right)$$

Onde $\text{Erros} = 0.1 \cdot \text{movimentos} + \text{custo} + 1000 \cdot \text{tempo} + 0.5 \cdot \text{revisita}$

E o custo ótimo é dado pelo algoritmo de busca clássica A^* .

- **Environment:** Labirinto caracterizado como um espaço determinístico, estático e discreto. Contém:

- Células livres;
- Paredes;
- Regiões desconhecidas;
- Posição inicial;
- Posição final.

- **Actuators:** Mover para Cima, Baixo, Esquerda, Direita.

$$A = \{\text{cima, baixo, esquerda, direita}\}$$

- **Sensors:** Acesso estrito apenas às células ou paredes adjacentes à posição do atual do agente.
- **Classificação:** Agente baseado em Modelo.
Como o ambiente é apenas parcialmente observável, um agente de reflexo simples faria loops infinitos ao deparar-se com becos sem saída. O sistema exige a manutenção de um estado interno do mundo (o modelo de transição). As estruturas de dados utilizadas para armazenar os nós explorados, o campo de visão atual e a pilha de retorno (*backtracking*) constituem o modelo mental do agente, viabilizando o raciocínio espacial e a fuga de impasses ao longo da navegação.

3 Formulação Formal dos Problemas

Os problemas de busca foram formulados da seguinte forma:

$$\{S, A, T, s_0, G, c\}$$

- S: conjunto de estados;
- A: conjunto de ações possíveis;
- T: modelo de transição, onde $T(s, a) = s'$;
- s_0 : posição inicial ($s_0 \in S$);
- G: teste de objetivo;
- c: custo de caminho.

3.1 Busca Clássica no Labirinto Conhecido

Nesta etapa, o agente conhece todo o labirinto desde o início. O problema é encontrar um caminho de A até B:

$$A \rightarrow B$$

- S: conjunto de células livres;
- A: {cima,baixo,esquerda,direita};
- T: $T((x, y), A) = (x', y')$;
- s_0 : posição inicial A;
- G: alcançar B;
- c: custo de cada movimento.

3.2 Busca Local com Pontos de Coleta

Nesta etapa, o agente tem acesso aos caminhos ótimos entre os pontos de coleta e os pontos de início e fim. O problema é a otimização combinatória da ordem de visitação dos pontos de coleta.

$$A \rightarrow C_{\pi(1)} \rightarrow C_{\pi(2)} \rightarrow \dots \rightarrow C_{\pi(k)} \rightarrow B,$$

onde π é uma permutação dos pontos de coleta.

- S: conjunto de todas as permutações válidas contendo o ponto inicial, os pontos de coleta e o ponto final;
- A: troca de pontos de coleta para gerar vizinhos;
- T: $T([C_{\pi(1)}, \dots, C_{\pi(k)}], A) = [C_{\pi'(1)}, \dots, C_{\pi'(k)}]$;
- s_0 : permutação inicial gerada de forma aleatória ou padronizada;
- G: atingir o número máximo de iterações ou alcançar um mínimo local/global;
- c: custo total de cada permutação dos pontos de coleta.

3.3 Busca Online no Labirinto Desconhecido

Nesta etapa, o agente tem acesso exclusivamente às células diretamente adjacentes a ele ou que já foram exploradas por ele. O problema é como chegar ao ponto B sem ter noção da localização do objetivo previamente.

$$A \rightarrow ??? \rightarrow B,$$

- S : conjunto de células livres;
- A : {cima, baixo, esquerda, direita};
- T : $T((x, y), A) = (x', y')$;
- s_0 : posição inicial A;
- G : alcançar B;
- c : custo acumulado de cada movimento realizado pelo agente.

4 Descrição dos Algoritmos

Para solucionar os problemas de busca definidos no domínio do labirinto, desenvolveu-se um conjunto de algoritmos para cada tipo de busca. Esta seção descreve o funcionamento interno das técnicas implementadas.

4.1 Busca Clássica no Labirinto Conhecido

4.1.1 Busca em Largura (BFS)

A Busca em Largura (BFS - *Breadth-First Search*) é um método de busca clássico que expande a fronteira de estados nível por nível. A implementação utiliza uma estrutura de dados do tipo fila (FIFO), garantindo que os nós descobertos primeiro sejam os primeiros a serem expandidos. Em cada iteração, o nó localizado na cabeça da fila é extraído e é testado se são as mesmas coordenadas do objetivo. Caso o estado atual corresponda ao estado final, o algoritmo retrocede pelos nós pais para reconstruir o caminho. Caso ainda não seja o objetivo, o nó é adicionado ao conjunto de estados explorados e é expandido. Seus vizinhos são adicionados à fila caso já não tenham sido explorados e o ciclo continua.

4.1.2 Busca em Profundidade (DFS)

A Busca em Profundidade (DFS - *Depth-First Search*) é um método de busca clássico que mergulha até o nó mais profundo possível até encontrar o objetivo ou realiza o retrocesso (*backtracking*). O algoritmo funciona com base em uma pilha (LIFO). A cada iteração, o agente extrai o estado mais recentemente descoberto, testa se é objetivo e em caso negativo, expande seus sucessores.

4.1.3 Busca Baseada em Prioridade (*Best-First Search*)

Os algoritmos de busca clássicos baseados em uma função Busca de Custo Uniforme (UCS), Busca Gulosa e Busca A^* foram implementados através de uma arquitetura central unificada denominada Busca por Prioridade. O algoritmo utiliza uma fila de prioridade do tipo *Min-Heap*, que delega a ordenação da fronteira de estados a uma função de avaliação genérica $f(n)$. Ao adotar este padrão de abstração, as estratégias de busca diferem unicamente pela formulação matemática utilizada na Busca por Prioridade:

- **Busca de Custo Uniforme (UCS):** A função de prioridade avalia estritamente o custo real acumulado da origem até o nó atual, caracterizando-se pela equação $f(n) = g(n)$.
- **Busca Gulosa:** A tomada de decisão é guiada exclusivamente pela heurística, estimando exclusivamente o custo do estado atual até o objetivo. Define-se pela equação $f(n) = h(n)$.
- **Busca A^* :** A função de prioridade soma o custo real já percorrido com a estimativa da heurística. Para dar maior flexibilidade, o algoritmo recebe um peso multiplicador (w) ajustável, resultando na equação $f(n) = g(n) + w \cdot h(n)$. Quando $w = 1.0$, o motor executa o A^* clássico. Se o valor de w for maior que 1.0, o algoritmo prioriza a heurística.

4.2 Busca Local com Pontos de Coleta

4.2.1 Hill Climbing

O algoritmo *Hill Climbing* foi utilizado para otimizar iterativamente a ordem de visita das coordenadas. A cada iteração o espaço de vizinhança é gerado trocando todos os pares de pontos de coleta da rota atual, isso gera uma combinação de vizinhos adjacentes.

A partir destes vizinhos a escolha é feita pelo somatório do custo real de locomoção no labirinto, calculado pelo algoritmo de busca clássica A^* que traça o caminho exato da *Origem* $\rightarrow$ *Coletas* $\rightarrow$ *Destino*.

O vizinho que apresentar a maior redução do custo total em relação ao custo atual é escolhido e o ciclo continua até alcançar o número máximo de iterações ou quando nenhum vizinho apresenta um custo menor que o estado atual.

4.2.2 Reinício Aleatório

O algoritmo é baseado no *Hill Climbing* mas, para que ele não fique estagnado em um mínimo local foi implementado um novo parâmetro que define o número de vezes que o algoritmo gera um ponto de início para a busca. O sistema gera múltiplas permutações iniciais das coordenadas de coleta, instanciando a busca em regiões independentes e distantes no espaço de soluções.

A partir de cada um desses pontos de partida, o agente executa o *Hill Climbing* até convergir para o mínimo local daquela região específica.

A execução do algoritmo é finalizada quando todas as instâncias independentes atingem seus respectivos mínimos locais ou quando o teto de iterações máximas é atingido. Dessa forma, a permutação selecionada ao final representa a melhor aproximação do ótimo global descoberta durante todo o ciclo exploratório.

4.2.3 Simulated Annealing

O *Simulated Annealing* introduz um parâmetro de "temperatura" (T) que controla a probabilidade de o agente aceitar transições para estados piores.

A cada iteração, o algoritmo seleciona de forma aleatória apenas um vizinho do espaço de permutações gerado, em vez de avaliar toda a vizinhança simultaneamente. A transição de estado obedece à seguinte lógica: se o vizinho selecionado apresentar um custo menor ($\Delta < 0$), a transição é aceita incondicionalmente. Se o vizinho for pior ($\Delta \geq 0$), a transição ainda pode ocorrer com uma probabilidade dada pela fórmula: $P = e^{-\Delta E/T}$.

À medida que a execução avança, a temperatura sofre um decaimento dado por um fator de resfriamento. Esse decréscimo reduz gradativamente a tolerância a movimentos prejudiciais, transicionando o comportamento do algoritmo lentamente.

A implementação guarda a permutação de menor custo absoluto alcançada. Isso garante que a aceitação de estados piores durante a fase de alta temperatura não sobrescreva ou apague o melhor resultado histórico descoberto pelo agente.

4.3 Busca Online no Labirinto Desconhecido

Na busca online foi implementado apenas um algoritmo, o algoritmo de Online DFS (*Depth-First Search*) que como na busca clássica vai mergulhando nos nós até encontrar o objetivo ou ficar sem saída e retroceder porém, o agente não tem conhecimento do mapa previamente.

Para viabilizar a navegação sob o mapa desconhecido sem que o agente caia em *loops* infinitos o agente atualiza após cada passo seu mapa conhecido registrando as coordenadas dos nós que visitou e dos nós adjacentes que são possíveis de ver. Caso um vizinho esteja livre e não tenha sido visitado o agente move-se para ele, o coloca na lista de retorno, caso seja necessário retornar após um beco sem saída, adiciona ao custo e à lista de visitados.

O algoritmo encerra sua execução de forma bem-sucedida ao chegar na coordenada do objetivo, ou falha caso a pilha de retorno seja esvaziada.

5 Metodologia Experimental

5.1 Os Labirintos Utilizados

Foram feitos quatro labirintos principais para a execução dos testes:

1. *easy.txt*: Labirinto simples para testar o funcionamento da busca rapidamente;

2. *big.txt*: Labirinto massivo pra testar a capacidade e tempo de busca de cada algoritmo;
3. *multiple.txt*: Tentativa de fazer um labirinto com múltiplas formas de solução, a ideia é que com múltiplas formas de solução seja mais fácil ver a discrepância entre os algoritmos;
4. *coleta.txt*: Mapa exclusivo para a busca local com pontos de coleta.

5.2 Bibliotecas e Ferramentas

Para a fundamentação estrutural e o equacionamento matemático dos algoritmos de busca, o projeto fez uso extensivo de módulos nativos da linguagem Python:

- ***Collections***: A estrutura *deque* foi especificamente adotada para a implementação de uma fila FIFO na busca por largura, visto que permite a remoção de itens em ambas as extremidades com menor complexidade algorítmica.
- ***Heapq e Itertools***: A estrutura foi empregada na construção de uma fila de prioridade para algoritmos de busca clássica. Foi utilizado o *Itertools* e sua função *count* para repassar um identificador sequencial junto ao nó, servindo como critério de desempate para custos iguais definidos pelo algoritmo em execução. ***Math***: Forneceu principalmente um limite numérico de custo inicial para os algoritmos e também foi utilizada para obter a função exponencial utilizada em *Simulated Annealing*.
- ***Random***: A biblioteca *random* foi utilizada para três propósitos embaralhar a sequência dos pontos de coleta nos algoritmos locais, escolher vizinhos aleatoriamente e definir a aceitação de piora temporários no algoritmo de *Simulated Annealing*.
- ***Typing e Dataclasses***: A biblioteca *typing* foi primariamente utilizada para evitar a tipagem dinâmica que ocorre na linguagem *Python* que pode causar problemas. *Dataclasses* foi utilizado para padronizar os resultados obtidos pelos algoritmos.
- ***Time***: A função *perf_counter* foi utilizada como um cronômetro para as execuções dos algoritmos de busca. Também foi utilizada a função *sleep* para dar um certo tempo de delay na animação do algoritmo de busca *online*.
- ***Sys***: De forma similar à biblioteca *time* foi utilizada na animação da busca *online* renderizando e apagando os quadros para dar a ilusão de animação no terminal.
- ***Pandas***: Utilizado para padronizar os resultados em *DataFrames* e exportá-los para arquivos do formato *.CSV*.
- ***Matplotlib e Seaborn***: Utilizados para plotagem gráfica, *Matplotlib* gera o gráfico e *Seaborn* o formata.

5.3 Modelagem dos resultados

A modelagem das estruturas de armazenamento de dados no repositório foi projetada utilizando o recurso de *dataclasses* com o objetivo de padronizar e isolar a coleta de métricas experimentais de acordo com a natureza de cada ambiente de busca.

- **ResultadoBusca (Clássica)**: Avalia o esforço do planejamento *offline*, mensurando o crescimento da fronteira de memória e a contagem de nós expandidos na árvore de busca.
- **ResultadoBuscaLocal (Local)**: Foca na otimização iterativa e combinatória. Substitui as métricas de expansão de grafos pelo monitoramento das iterações, a ordem da permutação gerada e, principalmente, a curva de convergência do algoritmo.
- **ResultadoBuscaOnline (Online)**: Foca no peso do movimento físico e observabilidade parcial. Quantifica as ações reais do agente células reveladas do labirinto desconhecido, revisitação de células e um histórico para a renderização da animação da trajetória.

6 Resultados

6.1 Busca Clássica no Labirinto Conhecido

Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo	Fronteira
BFS	Sim	121,0	121	136	0,00039	2
DFS	Sim	121,0	121	155	0,00036	5
UCS	Sim	121,0	121	136	0,00038	2
Greedy BFS	Sim	121,0	121	129	0,00037	2
A* (w=2.0)	Sim	121,0	121	129	0,00039	2

Tabela 1: Resultados dos Algoritmos Clássicos no Labirinto **easy.txt**

Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo	Fronteira
BFS	Sim	954,0	954	28528	0,0690	94
DFS	Sim	1104,0	1104	8372	0,0205	472
UCS	Sim	954,0	954	28528	0,0847	94
Greedy BFS	Sim	954,0	954	4897	0,0337	379
A* (w=2.0)	Sim	954,0	954	9228	0,0304	233

Tabela 2: Resultados dos Algoritmos Clássicos no Labirinto **big.txt**

Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo	Fronteira
BFS	Sim	525,0	525	8263	0,0196	53
DFS	Sim	599,0	599	4266	0,0104	253
UCS	Sim	525,0	525	8263	0,0241	53
Greedy BFS	Sim	525,0	525	4279	0,0137	188
A* (w=2.0)	Sim	525,0	525	5469	0,0177	106

Tabela 3: Resultados dos Algoritmos Clássicos no Labirinto **multiple.txt**

6.2 Busca Local com Pontos de Coleta

Algoritmo	Melhor Custo	Pior Custo	Custo Médio	Tempo Médio (s)	Iterações	Sucesso (%)
Simple Hill Climbing	304,0	432,0	321,0666	0,1159	3,0	86,66
Random Restart (k=5)	304,0	304,0	304,0	0,5663	14,0	100,0
Simulated Annealing	304,0	432,0	325,3333	0,6614	271,0	83,33

Tabela 4: Resultados dos Algoritmos Locais no Labirinto **coleta.txt**

6.3 Busca Online no Labirinto Desconhecido

Labirinto	Encontrado	Tempo (s)	Movimentos	Custo Real	Reveladas	Revisitadas
easy.txt	Sim	0,0008	133	133,0	288	6
big.txt	Sim	62,0154	26544	26544,0	25114	12460
multiple.txt	Sim	1,1423	7723	7723,0	7845	3447

Tabela 5: Resultados da Busca Online para Diferentes Labirintos

7 Gráficos

7.1 Busca Clássica no Labirinto Conhecido

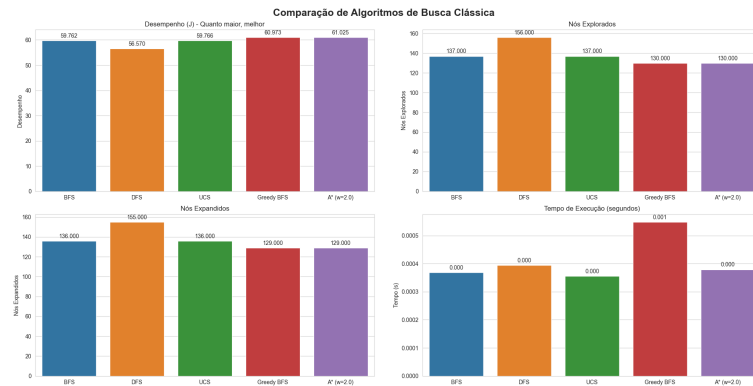


Figura 1: Comparação dos Algoritmos de Busca Clássica no Labirinto **easy.txt**

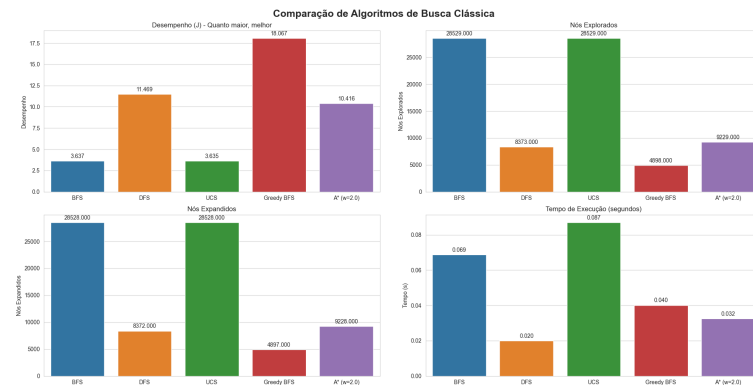


Figura 2: Comparação dos Algoritmos de Busca Clássica no Labirinto **big.txt**



Figura 3: Comparação dos Algoritmos de Busca Clássica no Labirinto **multiple.txt**

7.2 Busca Local com Pontos de Coleta

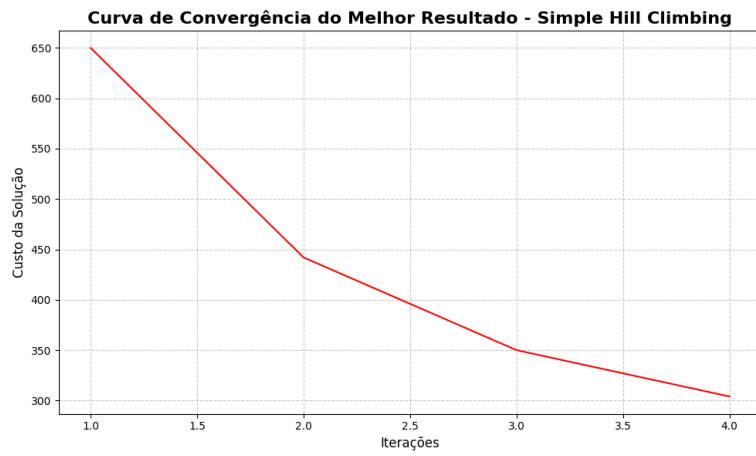


Figura 4: Curva de convergência do *Hill Climbing*



Figura 5: Curva de convergência do Reinício Aleatório

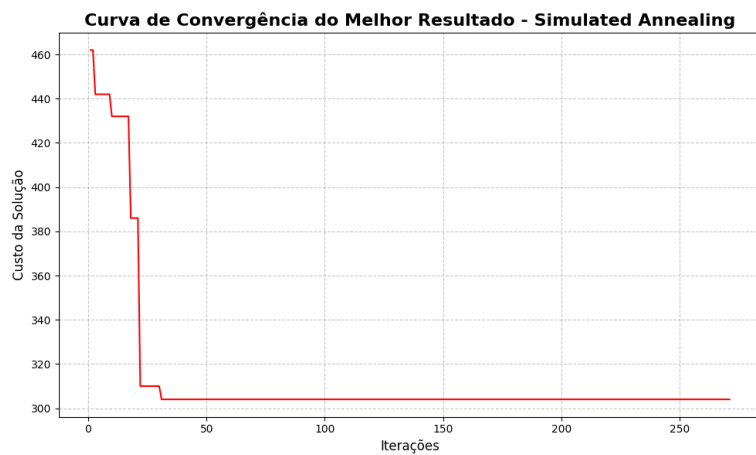


Figura 6: Curva de convergência do *Simulated Annealing*

8 Análise Crítica

8.1 Busca Clássica no Labirinto Conhecido

1. **BFS encontrou o menor caminho? Em quais condições isso ocorre?**
Sim. A Busca em Largura (BFS) garante a otimalidade de forma estrita em ambientes onde o custo de transição entre todos os nós é uniforme (peso 1 por passo no labirinto).
2. **DFS encontrou solução rapidamente? A solução foi boa?**
A DFS pode encontrar soluções rapidamente consumindo menos memória, mas a qualidade da solução é pior, gerando caminhos longos e excessivamente sinuosos devido à expansão profunda.
3. **UCS diferiu de BFS quando os custos variaram?**
Como os labirintos apresentam passos de custo unificado UCS e BFS comportam-se de forma funcionalmente idêntica. A UCS só divergiria se houvesse áreas do labirinto com pesos distintos.
4. **A busca gulosa foi eficiente? Foi ótima?**
Foi altamente eficiente, porém não foi ótima, guiando-se apenas pela heurística e sacrificando rotas curtas em troca de velocidade na direção geral do alvo.
5. **A* equilibrou qualidade da solução e eficiência?**
Ao somar o custo real percorrido e a estimativa do objetivo ($f(n) = g(n) + h(n)$), o A^* encontrou o caminho ótimo assim como a UCS, mas expandindo uma fração muito menor do espaço de estados.
6. **A heurística utilizada é admissível? Justifique.**
Sim, a Distância de Manhattan é admissível. Como o agente só se move nos 4 eixos ortogonais e não cruza paredes na diagonal, a heurística jamais superestima o custo físico real para o objetivo.

8.2 Busca Local com Pontos de Coleta

1. **Hill-Climbing ficou preso em mínimo local?**
Sim. Como a implementação exige melhoria estrita a cada movimento, o algoritmo estagnou assim que atingiu o primeiro mínimo local.
2. **Simulated Annealing encontrou soluções melhores?**
Sim. Ao aceitar transições probabilísticas para estados piores de acordo com a probabilidade, o método conseguiu sair de mínimos locais, alcançando permutações superiores.
3. **Como a temperatura inicial influenciou os resultados?**
Temperaturas altas garantem exploração mais global no início da execução, permitindo ao algoritmo ignorar movimentos ruins e mapear mínimos melhores e distantes no espaço de permutações.
4. **Como a taxa de resfriamento afetou a convergência?**
Um decaimento lento retarda a convergência e eleva a chance do ótimo global, enquanto resfriamentos bruscos forçaram o algoritmo à exploração gulosa.
5. **A busca local encontrou sempre a solução ótima?**
Não. Por serem combinatórias, esses métodos entregam aproximações mas não oferecem garantias da solução ótima.
6. **Qual foi o compromisso entre tempo e qualidade?**
A depender do método, como *Random Restart* há a multiplicação do tempo de processamento em troca de custo global mais otimizado.

8.3 Busca Online no Labirinto Desconhecido

1. **O agente online tomou decisões subótimas? Por quê?**
Sempre. Decisões subótimas estão fadadas a ser tomadas devido ao modelo de observabilidade parcial. O agente entra em corredores errados porque não enxerga que não tem saída adiante.
2. **Quais informações faltavam ao agente?**
Faltava o acesso ao mapa completo, localização do objetivo e heurística.

3. **O mapa interno convergiu para o mapa real?**

Não, pois o agente explora até chegar ao objetivo, ou seja, ele pode tanto explorar uma porcentagem mínima do mapa e chegar rapidamente ao objetivo ou pode explorar o mapa todo e chegar ao objetivo por último, apenas nesse último caso o mapa convergiria para o mapa real.

4. **O agente revisitou muitas células?**

Sim. Devido à mecânica de *backtracking* cada beco sem saída forçou o agente a caminhar para trás.

5. **Como melhorar a exploração?**

Implementando algoritmos baseados em aprendizado onde o agente atualiza o valor heurístico da célula ao deixá-la, "lembrando" de evitar becos sem saída conhecidos no futuro.

6. **O que diferencia busca online de busca clássica?**

A Busca Clássica descarta rotas ruins instantaneamente. A Busca *Online* exige execução intercalada se o agente tomar uma decisão equivocada ele paga em custo.

9 Limitações

Labirintos Perfeitos

A geração dos labirintos utilizados tendeu a criar labirintos perfeitos isto é, que só apresentam um único caminho para alcançar o objetivo. Essa estrutura restringe a avaliação do melhor custo, forçando algoritmos naturalmente subótimos (como a Busca em Profundidade e a Busca Gulosa) a convergirem para o mesmo custo final do algoritmo A^* . Caso o ambiente possuísse espaços mais abertos e múltiplos corredores viáveis, a discrepância na qualidade das rotas geradas pelos diferentes algoritmos ficaria muito mais evidente na função de desempenho.

Ausência de Base Comparativa na Busca *Online*

A implementação de algoritmos para a busca *online* limitou-se a um único algoritmo exploratório (DFS *Online*). Essa restrição impossibilita uma análise comparativa relativa ao desempenho do agente. A implementação paralela de algoritmos guiados em tempo real permitiria comparar a eficiência do DFS com estratégias alternativas que também exploram o labirinto cegamente.

10 Uso de Inteligência Artificial

Gemini Versão 3.1 Pro: Utilizada para ajudar na formatação do texto em LaTeX, codificação do projeto, dúvidas na teoria das buscas, correção de erros.

GitHub Copilot 0.48.1: Utilizado no *VSCODE* para autocompletar blocos de código repetitivos, sugerir fechamentos sintáticos e agilizar a digitação de chamadas de funções e importações durante o desenvolvimento.

10.1 Principais Prompts Utilizados

- Aperfeiçoamento de código ou explicação no relatório técnico: "Verifique, corrija e aperfeiçoe [trecho de código/explicação sobre algoritmo] : [código/texto]";
- Duvidas sobre a implementação dos algoritmos de busca ainda não implementados em aula;
- Aperfeiçoamento de trechos de código que podem ser substituídos por funções nativas;
- *Debugging* e ajuda na correção de erros: "O que tem de errado na função X?";
- Tentativa de fabricação de labirintos por meio da IA;
- Exportação de dados: "Como faço para concatenar esses objetos e gerar uma tabela CSV?";
- Dúvida de de conceitos teóricos: "Resuma o funcionamento do algoritmo de busca *Simulated Annealing*";
- Resolução de problemas de formatação no LaTeX: "Como faço para colocar uma imagem/tabela no LaTeX?";
- Funcionamento de bibliotecas: "Como funciona a biblioteca X? Quais os parâmetros do comando Y?"

10.2 Trechos de Código Sugeridos por IA

- Exportação via Pandas: Como já havia utilizado pandas e não sabia como exportar os resultados para .CSV a IA sugeriu a conversão direta pelo pandas que foi bem fácil de utilizar;
- Renderização do Labirinto (Animação): Ao indagar com a IA sobre como fazer uma animação para Busca Online ela retornou com sugestões de código que foram adotadas na função `animar_busca_online` no que antes eram chamadas diretas de `print`;
- Adaptação do código de aulas anteriores: O código de aulas anteriores continha as funções de busca clássicas já implementadas e junto com a ajuda da IA adaptamos essas funções para o trabalho atual;
- Arquitetura de código: Indaguei com a IA qual seria a melhor arquitetura para rodar o código e chegamos na atual onde o `main` utiliza os agente para executar as funções de busca.

10.3 Sugestões Rejeitadas

- Geração de Labirinto: A IA fez um labirinto de caracteres ASCII para testes do algoritmo de busca porém a exportação ou escrita dele estava quebrada e não consegui utilizá-lo;
- Animação em janela ao invés de terminal: Na função `animar_busca_online` a IA sugeriu o uso de uma janela a parte para executar a animação porém devido à sua complexidade e já estar avançado no projeto optei por não mudar a função.

10.4 Erros Cometidos por IA

- Geração de Labirinto quebrada;
- Mudança de nome de variáveis ao corrigir ou gerar códigos;
- Excesso de complexidade na execução de algoritmos de busca a fim de economizar memória.

10.5 Como o Grupo Validou a Solução

- Execução: Funções sugeridas foram executadas e os relatórios de erro foram analisados e devolvidos iterativamente até a obtenção de um código funcional.
- Validação Teórica: As explicações algorítmicas geradas pela IA para o código ou relatório foram cruzadas com as exigências do trabalho e com o material disponibilizado pelo professor para garantir que os conceitos estavam sendo seguidos.

10.6 Modificações Feitas pelo Grupo

- Correção de atributos trocados no código sugerido (ex: as variáveis `custo_percorrido` e `n_movimentos` que a IA havia invertido nos resultados das buscas).
- Arredondamento de strings de ponto flutuante no relatório em LaTeX para evitar que os resultados das tabelas extrapolassem as margens do documento no PDF final.
- Modificação dos nomes dos atributos das partes de código geradas por IA para condizer com o restante do trabalho.

11 Conclusão

Este trabalho cumpriu com o objetivo de modelar, implementar e analisar o comportamento de um agente inteligente submetido aos três tipos de busca. A mudança entre os ambientes mostrou como a percepção dita a abordagem de cada busca.

Em suma, as métricas e gráficos levantados atestam que não existe um algoritmo universalmente superior. A concepção de um agente inteligente eficaz exige um alinhamento entre a configuração do espaço de estados, as restrições sensoriais do ambiente e a formulação de sua função de desempenho.